SKILL WEEK - 10

Task 1 - To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior



```c
#include <stdlib.h>
#include <unistd.h>
#include <limits.h>
#include <string.h>

int main()
{
    char current_dir[PATH_MAX];
    char command[100];

    printf("Simple Shell State Program\n");
    printf("Type 'pwd' to display current directory.\n");
    printf("Type 'exit' to terminate.\n\n");

    while (1)
    {
        /* Retrieve current directory */
        if (getcwd(current_dir, sizeof(current_dir)) == NULL)
        {
            perror("getcwd");
            break;
        }

        /* Display current path */
        printf("%s$ ", current_dir);
        fflush(stdout);

        /* Read command */
        if (fgets(command, sizeof(command), stdin) == NULL)
        {
            printf("\nEnd of input.\n");
            break;
        }

        command[strcspn(command, "\n")] = '\0';

        /* Display current directory */
        if (strcmp(command, "pwd") == 0)
        {
            printf("Current Directory: %s\n", current_dir);
        }

        /* Exit request */
        else if (strcmp(command, "exit") == 0)
        {
            printf("Exit request received.\n");
            printf("Cleaning up resources...\n");
            printf("Saving shell state...\n");
            printf("Program terminated successfully.\n");
            break;
        }

        /* Empty command */
        else if (strlen(command) == 0)
        {
            continue;
        }

        /* Invalid command */
        else
        {
            printf("Unknown command: %s\n", command);
        }
    }

    return 0;
}
```

```
satwika@SATWIKAG: ~/tas
satwika@SATWIKAG:~$ mkdir task1_shell_state
cd task1_shell_state
satwika@SATWIKAG:~/task1_shell_state$ nano shell_state.c
satwika@SATWIKAG:~/task1_shell_state$ gcc shell_state.c -o shell_state
satwika@SATWIKAG:~/task1_shell_state$ ./shell_state
Simple Shell State Program
Type 'pwd' to display current directory.
Type 'exit' to terminate.

/home/satwika/task1_shell_state$ pwd
Current Directory: /home/satwika/task1_shell_state
/home/satwika/task1_shell_state$ getcwd()
Unknown command: getcwd()
/home/satwika/task1_shell_state$ hello
Unknown command: hello
/home/satwika/task1_shell_state$ exit
Exit request received.
Cleaning up resources...
Saving shell state...
Program terminated successfully.
satwika@SATWIKAG:~/task1_shell_state$
```

Task 2 - To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 200

/* Check whether variable name is valid */
int valid_name(const char *name)
{
    if (name == NULL || name[0] == '\0')
        return 0;

    if (!isalpha((unsigned char)name[0]) && name[0] != '_')
        return 0;

    for (int i = 1; name[i] != '\0'; i++)
    {
        if (!isalnum((unsigned char)name[i]) && name[i] != '_')
            return 0;
    }

    return 1;
}

/* Handle export command */
void handle_export(char *input)
{
    char *assignment = input + 7;

    while (*assignment == ' ')
        assignment++;

    char *equals = strchr(assignment, '=');

    if (equals == NULL)
    {
        printf("Usage: export NAME=value\n");
        return;
    }

    *equals = '\0';
```

```c
    char *name = assignment;
    char *value = equals + 1;

    if (!valid_name(name))
    {
        printf("Invalid variable name: %s\n", name);
        return;
    }

    if (setenv(name, value, 1) == -1)
    {
        perror("setenv");
        return;
    }

    printf("Exported: %s=%s\n", name, value);
}

/* Show variable value */
void handle_print(char *name)
{
    char *value = getenv(name);

    if (value != NULL)
        printf("%s=%s\n", name, value);
    else
        printf("%s is not set\n", name);
}

/* Test export in a child process */
void test_child_process()
{
    pid_t pid = fork();

    if (pid == -1)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        char *value = getenv("MYVAR");
```

```
        if (value != NULL)
            printf("Child process: MYVAR=%s\n", value);
        else
            printf("Child process: MYVAR is not set\n");

        exit(0);
    }
    else
    {
        wait(NULL);
        printf("Parent process: Child completed.\n");
    }
}

int main()
{
    char input[MAX_INPUT];

    printf("Simple Export Shell\n");
    printf("Commands:\n");
    printf("  export NAME=value\n");
    printf("  print NAME\n");
    printf("  child\n");
    printf("  exit\n\n");

    while (1)
    {
        printf("export-shell$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

        if (strncmp(input, "export ", 7) == 0)
        {
            handle_export(input);
```

```
satwika@SATWIKAG:~/task1_shell_state$ mkdir task2_export
cd task2_export
satwika@SATWIKAG:~/task1_shell_state/task2_export$ nano export_shell.c
satwika@SATWIKAG:~/task1_shell_state/task2_export$
satwika@SATWIKAG:~/task1_shell_state/task2_export$ gcc export_shell.c -o export_shell
satwika@SATWIKAG:~/task1_shell_state/task2_export$ ./export_shell
Simple Export Shell
Commands:
  export NAME=value
  print NAME
  child
  exit

export-shell$ export MYVAR=Hello
Exported: MYVAR=Hello
export-shell$ print MYVAR
MYVAR=Hello
export-shell$ export MYVAR=Welcome
Exported: MYVAR=Welcome
export-shell$ print MYVAR
MYVAR=Welcome
export-shell$ setenv(name, value, 1);
Unknown command: setenv(name, value, 1);
export-shell$ export 123ABC=hello
Invalid variable name: 123ABC
export-shell$ export MY-VAR=hello
Invalid variable name: MY-VAR
export-shell$ export MYVAR=Hello
Exported: MYVAR=Hello
export-shell$ child
Child process: MYVAR=Hello
Parent process: Child completed.
export-shell$ print TEST
TEST is not set
export-shell$ exit
Exiting shell...
satwika@SATWIKAG:~/task1_shell_state/task2_export$
```